

FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE
COURSE: ETL (G01)

Lab 1B – From Business Requirements to a Basic ETL Pipeline

1. Introduction

In Lab 1A, your team identified the business objectives, analytical questions, data requirements, KPIs, and expected outputs for a retail analytics platform. In this laboratory, you will build the first working version of the data system required to satisfy those needs.

The focus is not on advanced transformations or optimization. The focus is on systems thinking: designing the pipeline as a set of connected blocks, implementing each block independently, and verifying that all components work together as one end-to-end system.

In real-world Data Engineering projects, technical implementation is never the first step. Before designing databases, writing code, or building ETL pipelines, engineers must first understand the business problem they are trying to solve.

2. Learning Objectives

Implement a basic ETL pipeline that integrates heterogeneous retail data sources and produces a structured analytical repository capable of answering the business questions defined in Lab 1A.

By the end of this lab, students will be able to:

- Translate business requirements into technical pipeline components.
- Represent an ETL system using a block diagram before coding.
- Extract data from CSV, JSON, and XML sources.
- Perform basic profiling, cleaning, schema harmonization, integration, and transformation.
- Load processed data into a SQLite database.
- Execute analytical SQL queries that verify whether the business requirements were satisfied.
- Explain how each block contributes to the behavior of the complete system.

3. Dataset Description

The data cover three months (February–April 2026) and three branches. Transaction data are intentionally distributed across different formats to simulate heterogeneous operational systems.

File	Format	Purpose
sales_cali.csv	CSV	Sales transactions from Cali.
sales_bogota.json	JSON	Sales transactions from Bogotá.
sales_medellin.xml	XML	Sales transactions from Medellín.

File	Format	Purpose
products.csv	CSV	Product names, categories, list prices, and costs.
stores.csv	CSV	Store, city, and region information.
promotions.csv	CSV	Promotion dates, discount percentages, and campaigns.
monthly_targets.csv	CSV	Monthly sales target for each branch.

The transaction sources contain a small number of realistic quality issues. Students are expected to discover them through profiling rather than receiving a predefined list.

4. Dataset Description

Before implementing code, represent the solution using the following blocks:

1. Business requirements
2. Data sources
3. Extract
4. Profile
5. Clean and harmonize
6. Transform and integrate
7. Validate
8. Load
9. Query and evaluate business requirements

Your diagram must show the input and output of every block and the connection between blocks. Do not begin coding until the team can explain how the complete system should work.

5. Recommended Project Structure

```

Lab1B_ETL/
|
|— data/
|   |— raw/
|   |— processed/
|   |— output/
|
|— database/
|   |— retail_analytics.db
|
|— src/
|   |— extract.py
|   |— transform.py
|   |— load.py
|   |— queries.py
|   |— main.py

```

```

|
|— docs/
|   |— pipeline_diagram.png
|
|— README.md
|— requirements.txt
|— .gitignore

```

6. Activities

Activity 1 – Review and Trace the Requirements

Review your Lab 1A deliverable and select the six highest-priority business requirements that will be implemented in the first version of your analytical platform.

For each requirement, identify the data needed, the expected transformation, and the expected analytical output. These requirements will guide the design and implementation of your ETL pipeline.

Business Requirement	Required Data	Pipeline Block	Expected Output
Monitor total revenue and seasonal trends	POS / Transactional System (sales_cali.csv, sales_bogota.json, sales_medellin.xml)	Extract → Clean → Aggregate by date/store → Load	Aggregate table of daily and monthly sales by store and city
Identify sales performance by category	POS System + Inventory Database / Catalog (products.csv)	Extract → JOIN sales with products on product_id → Group by category	Performance dataset by category (units, revenue, rank, % of total)
Evaluate goal achievement by store and cities	POS System + ERP / Presupuestos (monthly_targets.csv)	Aggregate net_sales by store+month → JOIN with monthly_targets → Calculate compliance %	Table of Compliance Facts (actual vs target, % compliance, status)
Monitor the impact of promotions	POS System + Sistema de Promociones (promotions.csv)	JOIN sales with promotions by promotion_code and date range → Calculate ROI and sales lift	Campaign effectiveness tables (ROI, discount cost, lift vs baseline)

Activity 2 – Design the Pipeline

Create a block diagram showing all data sources, processing stages, outputs, and connections. For each block, define: purpose, input, processing responsibility, output, and possible failure.

Block	Input	Responsibility	Output	Possible Failure
Extract	Raw files: sales_cali.csv, sales_bogota.json, sales_medellin.xml	extract.py — reads each source in its native format and maps columns to the	Consolidated raw DataFrame (763 rows) with unified column names	FileNotFoundError if a source file is missing; malformed JSON/XML

	; reference CSVs (products, stores, promotions, monthly_targets)	common schema (COMMON_TRANSACTION_COLUMNS)		causing parse failure; wrong encoding
Profile	Consolidated raw DataFrame from Extract	transform.py::profile_raw_data() — counts nulls per column, duplicate IDs, invalid quantities/prices, unparseable dates, unique categoricals	Profiling summary dictionary printed to console (audit evidence)	Silent false negatives if data types are unexpected; profiling is read-only so no blocking failure
Clean/Harmonize	Raw DataFrame + profiling findings	transform.py::clean_and_harmonize_data() — strips whitespace, uppercases IDs, converts dates (mixed format), casts numeric columns, fills promotion_code nulls with "NONE", drops invalid rows	Clean DataFrame (755 valid rows, down from 763)	Overly aggressive filtering drops valid records; format="mixed" may silently misparse ambiguous dates (e.g. 04-08-2026)
Transform/Integrate	Clean DataFrame + reference tables (products, stores, promotions)	transform.py::transform_and_integrate_data() — LEFT JOINS with master tables on product_id, store_id, promotion_code; calculates gross_sales, discount_amount, net_sales; extracts year/month/week/day_name	Fully integrated analytical DataFrame (755 rows, ~24 columns) with financial metrics	Unmatched foreign keys leave NaN in store_name/product_name (caught by assertions); incorrect discount_rate cascades into wrong net_sales
Validate	Integrated DataFrame	Assert statements inside transform_and_integrate_data(): uniqueness of sale_line_id, no nulls in net_sales, no negative net_sales, no unmapped store_name or product_name	Pipeline halts with descriptive AssertionError if any invariant is violated	Assertions raise uncaught exceptions that stop the pipeline — intentional fail-fast behavior
Load	Validated integrated DataFrame + reference DataFrames	load.py — saves integrated_sales.csv to data/processed/; writes sales_analytics table to SQLite; writes all	retail_analytics.db with tables: sales_analytics, products, stores, promotions, monthly_targets	Disk full or permission error; if_exists="replace" means a partial run deletes the previous

		reference tables to SQLite		good data before the new write completes
Query	retail_analytics.db (SQLite)	queries.py — executes 7 SQL queries covering REQ 1–4; saves each result as a named table (REQ1_daily_sales_aggregate, REQ2_category_performance, REQ3_compliance_facts, REQ4_campaign_effectiveness, etc.)	7 analytical tables in SQLite + printed console report verifying all 4 business requirements	Missing table if a prior step failed; window functions (RANK, SUM OVER) require SQLite ≥ 3.25 — older versions raise OperationalError

Activity 3 – Extract

Implement one extraction function for each transaction format and functions for the reference tables. Every transaction source must be converted to a pandas DataFrame with a common schema.

Common Field	Meaning
sale_line_id	Unique sale-line identifier
sale_date	Transaction date
store_id	Store identifier
product_id	Product identifier
quantity	Units sold
unit_price	Price before discount
promotion_code	Applied promotion, when applicable
payment_method	Payment method

The extraction block must not perform business calculations. Its responsibility is to read each source and expose the data in a common technical structure. No cleaning, validation, or business transformations should be performed during the extraction stage.

Activity 4 – Profile the Extracted Data

Before cleaning or transforming, generate a compact profiling summary for the combined transaction data: row count, columns and data types, missing values, duplicate sale_line_id values, invalid quantities, invalid prices, invalid dates, and distinct values for selected categorical fields.

Record the findings in the README and explain which cleaning decisions are derived from each finding.

Activity 5 – Clean and Harmonize

Cleaning improves data quality. Apply only the rules supported by the profiling findings:

- Standardize column names and ID values.
- Trim whitespace and standardize text casing.
- Parse the different date formats into one date type.
- Convert quantity and unit_price to numeric values.
- Remove duplicated sale_line_id records, keeping the first valid occurrence.
- Reject records with invalid dates, quantity ≤ 0 , or unit_price ≤ 0 .
- Represent missing promotion codes consistently.

All cleaning decisions must be justified using the profiling results obtained in Activity 4.

Activity 6 – Transform and Integrate

Transformation changes data so that it can satisfy analytical requirements. Integrate the transactions with the reference tables and create:

- product_name and category from products.csv
- store_name, city, and region from stores.csv
- discount_pct and campaign_name from promotions.csv
- gross_sales = quantity $\times$ unit_price

- $\text{discount_amount} = \text{gross_sales} \times \text{discount_pct}$
- $\text{net_sales} = \text{gross_sales} - \text{discount_amount}$
- month, week, and day_name from sale_date
- sales_target from monthly_targets.csv

If you considered, you could add other columns to the integrated sales table.

Only transformations required to satisfy the selected business requirements should be implemented.

Activity 7 – Validate

Before loading, verify at minimum:

- sale_line_id is unique.
- Required identifiers and dates are not null.
- quantity, unit_price, gross_sales, and net_sales are positive.
- Every product matches the product master.
- Every store matches the store master.
- net_sales equals gross_sales minus discount_amount.

If a critical validation fails, the pipeline should stop before loading and record the reason in the log. Validation is the final quality gate before loading the analytical repository.

Activity 8 – Load

Save the processed dataset as data/processed/integrated_sales.csv and load it into a SQLite database named retail_analytics.db using a table named sales_analytics. The database is the analytical output of the pipeline; the final queries must read from SQLite, not from the raw files.

Activity 9 – Analytical Queries

Implement a simple menu or script that shows selected business requirements selected.

Each query must include a short explanation connecting the result to a business question from Lab 1A.

Activity 10 – Orchestrate the Complete System

Implement main.py as the entry point. It must call the blocks in the correct order, record start/end messages and row counts, stop on critical failures, and produce the CSV, SQLite database, and log file. Run the complete pipeline from the project root using one command.

Activity 11 – Verification of Requirements

Complete the table after executing the pipeline.

Business Requirement	Evidence Produced	Satisfied?	Explanation
Monitor total revenue and seasonal trends	REQ1_daily_sales_aggregate: 148 rows covering Feb–Apr 2026 across Cali Norte, Bogotá Centro, and Medellín Poblado, showing daily transactions, units sold, gross/net sales, and avg ticket per store. REQ1_monthly_trend: 13 rows with monthly rollup showing seasonal pattern (Feb peak → Mar dip → Apr recovery for Cali; Medellín peaks in Mar).	Yes	The pipeline successfully cleaned nulls (522 → 0 per key column), standardized 3 date formats, and separated sales by date and store. All 3 cities and all 3 months are represented with accurate daily and monthly aggregates.

Business Requirement	Evidence Produced	Satisfied?	Explanation
Identify sales performance by category	REQ2_category_performance: 4 rows ranking Small Appliances (44.78%, \$81.7M net), Electronics (25.96%, \$47.3M), Home & Office (16.94%, \$30.9M), Personal Care (12.31%, \$22.5M). REQ2_product_breakdown: 15 rows with each product ranked within its category (e.g. Air Fryer 4L #1 in Small Appliances with \$32.7M net).	Yes	JOIN between sales_analytics and products catalog (product_id) was successfully performed. Output provides a complete performance dataset by category including units sold, revenue, discounts, average ticket, and % share of total revenue.
Evaluate goal achievement by store and cities	REQ3_compliance_facts: 9 rows (3 stores × 3 months). Cali Norte achieved target in Feb (117.64%); Medellín Poblado achieved in Mar (110.65%). Bogotá Centro was consistently below target (79.87%, 51.20%, 56.22%). All stores shown with actual sales, monthly target, compliance %, and status (ACHIEVED /	Yes	Pipeline successfully joined POS data with ERP budget data (monthly_targets) on store_id and month. Compliance percentage and categorical status were calculated as required. The table covers all 3 cities and 3 months with no missing rows.

	NEAR TARGET / BELOW TARGET).		
Monitor the impact of promotions	REQ4_campaign_effectiveness: 6 rows with ROI per campaign (Home Office PROMO08: 124,800% ROI; Coffee Week & Wearables Week: 99,800%; Personal Care Month lowest at 49,800%). REQ4_sales_lift: 6 rows comparing promoted vs non-promoted sales for the same product in the same period (Kitchen Days lifted units +20%; Coffee Week showed -53.33% unit lift despite positive avg sale lift).	Yes	JOIN between sales and promotions was filtered by promotion date range (start_date to end_date). ROI, revenue per discount dollar, and sales lift vs baseline (same product, no promo, same period) were all calculated. Results cover all 6 campaigns with campaign-level and product-level granularity.

Activity 12 – Reflection Questions

1. How did the requirements from Lab 1A influence the design of the pipeline?

R/ The four business requirements from Lab 1A acted as the architectural blueprint for every design decision in Lab 1B. Each requirement directly determined which data sources to ingest (e.g., the need to monitor promotions meant the promotions.csv and its date range had to be incorporated), which transformations to apply (e.g., evaluating goal achievement required a JOIN with the ERP budget table monthly_targets.csv), and what the final output tables should look like (daily aggregates, compliance facts, campaign effectiveness). Without the Lab 1A requirements, the pipeline would have no criteria to determine which columns to preserve, which JOINS were necessary, or which metrics to calculate. The requirements also established the validation criteria: for instance, knowing that store performance needed to be compared city by city meant the pipeline had to assert that store_name was never null after the JOIN — otherwise a missing store mapping would silently corrupt the compliance report.

2. What is the difference between profiling, cleaning, transformation, and validation in your implementation?

R/ These four stages are deliberately separated in the codebase and serve distinct purposes. Profiling (profile_raw_data) is a passive diagnostic step: it reads the raw DataFrame and counts nulls, duplicates, invalid quantities, invalid prices, and unparseable dates without modifying anything. Its output is an audit report that informs the cleaning rules. Cleaning (clean_and_harmonize_data) is the first active step: it applies corrections based on profiling findings — stripping whitespace, standardizing casing, converting mixed-format dates, casting numeric columns, filling promotion_code nulls with "NONE", and filtering out records with invalid quantities or prices. The data at this stage is syntactically correct but not yet enriched. Transformation (transform_and_integrate_data) goes further: it joins the clean transactions with three master tables (products, stores, promotions) to add semantic context, and calculates business metrics (gross_sales, discount_amount, net_sales) and temporal attributes (year, month, week, day_name) that did not exist in the raw data. Validation, implemented as Python

assert statements at the end of transformation, verifies that the resulting dataset satisfies non-negotiable business invariants: `sale_line_id` must be unique, `net_sales` must have no nulls and no negatives, and every `store_id` and `product_id` must have successfully mapped to a name. If any assertion fails the pipeline halts immediately with a descriptive error, preventing corrupt data from reaching the load stage.

3. Why was it necessary to design the system as blocks before coding?

R/ Designing the pipeline as discrete blocks (Extract, Profile, Clean, Transform, Load, Query) before writing any code provided three critical benefits. First, it enforced separation of concerns: each block has a single responsibility, which meant that when Bogotá's JSON file used Spanish column names (`id_linea`, `sucursal`, `unidades`) instead of the English schema, only `extract.py` needed to be modified — the remaining five blocks were unaffected. Second, the block design made the system testable in isolation: the profiling report could be inspected after extraction without running the full pipeline, and each function could be called independently from the Python interpreter for debugging. Third, it aligned the technical architecture directly with the business requirements identified in Lab 1A: the Query block exists precisely because the requirements demanded verifiable analytical outputs, not just a clean CSV. Without this upfront design, the team would have likely written a monolithic script where cleaning, transformation, and loading were interleaved, making it nearly impossible to diagnose where a data quality issue originated or to swap one component without breaking the others.

4. Which block would be most affected if a branch changed its file format?

R/ The Extract block (`extract.py`) would be the only block requiring modification. This is a deliberate consequence of the modular architecture: the contract between Extract and all downstream blocks is the `COMMON_TRANSACTION_COLUMNS` schema (`sale_line_id`, `sale_date`, `store_id`, `product_id`, `quantity`, `unit_price`, `promotion_code`, `payment_method`). As long as `extract.py` maps the new format to this schema, the Profile, Clean, Transform, Load, and Query blocks require zero changes. This was demonstrated in practice during the lab: Medellín's XML used tags like `branch_code`, `sku`, `units`, and `unit_value`, while Bogotá's JSON used `id_linea`, `sucursal`, `unidades`, and `precio`. Adding `rename()` mappings in `extract.py` was sufficient to make all 755 records flow correctly through the remaining pipeline. If the format change introduced a new business concept (e.g., a new column representing a tax amount), then `transform.py` would also need to incorporate that new metric — but that would be an extension, not a repair.

5. Did the team build an ETL pipeline, or did it build a system to solve a business problem? Explain.

R/ The team built a system to solve a business problem, and the ETL pipeline was the mechanism chosen to implement it. The distinction matters: a generic ETL pipeline would simply move data from heterogeneous sources into a unified store. What the team built goes further — every design decision was driven by the four business requirements from Lab 1A. The decision to join with `monthly_targets` was not a data engineering preference; it was the direct answer to the business question "are our stores meeting their revenue goals?" The decision to calculate ROI and sales lift for promotions was not a technical exercise; it was the answer to "which marketing campaigns are actually driving incremental revenue?" The validation assertions (e.g., no unmapped `store_id`) exist not because they are good engineering practice in the abstract, but because a missing store mapping would make the compliance table misleading to a regional manager. Even the choice to separate Cali (CSV), Bogotá (JSON), and Medellín (XML) into distinct extraction functions reflects the business reality that each city is an independent operational unit with its own data management practices. The ETL pipeline is the infrastructure; the business problem — giving the retail chain's leadership reliable, comparable, and actionable analytics across three cities — is what the system was actually built to solve.